LearnCpp coding workbook

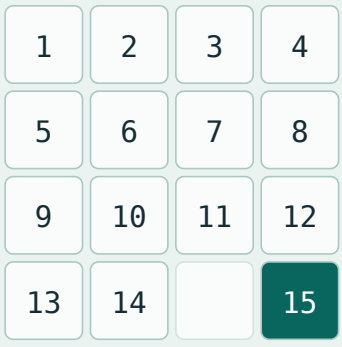
A chapter-ordered collection of C++ programming problems, from reading an integer to managing the lifetime of a polymorphic shape.

155	108	356
workbook entries	linked exercise pages	course pages reviewed

FROM THE CHAPTER 21 PROJECT

One move from solved

In this 15-puzzle, the input describes the direction a numbered tile travels. Which command slides tile 15 into its final position?



Open the full project ›

Using this workbook

Each entry has an implementation brief, a check you can run and a cost target. Tick a box as you finish. Select a lesson number to open the original problem, starter code and any published solution.

Browse by topic ›

Aman Choudhari

10 November 2026

WORKBOOK GUIDE

Contents

Select a topic or enter a physical page number in your PDF reader.

0–3	First programs and debugging	22 entries	3–7
4–6, O	Numbers, strings and operators	15 entries	8–10
7–9	Control flow and validation	16 entries	11–14
10–13, F	Types, functions and pointers	15 entries	15–17
14–15	Classes and object design	12 entries	18–20
16–19	Containers and algorithms	31 entries	21–27
20–21	Functions and operators	20 entries	28–31
21.y	The 15-puzzle	3 entries	32
22–25	Ownership and polymorphism	14 entries	33–35
26–28, C	Templates, exceptions and I/O	7 entries	36–37
Index	Linked directory of all course pages		38

What is counted

We reviewed the 356 pages in the LearnCpp table of contents on 10 November 2026. The 155 entries cover author-set coding, refactoring, debugging and test exercises, the chapter project and selected hands-on walkthroughs. Related subparts share an entry. The directory also links theory and output-prediction quizzes. Reader comments are outside the count.

Difficulty and cost

Easy Practice a single feature or fix a local error.
Intermediate Combine features, design an interface or reason through several cases.
Advanced Integrate multiple components or manage invariants across operations.

Difficulty, worked checks and cost estimates are editorial additions. Time describes a typical implementation with fixed-width scalar operations and constant-time scalar formatting. Space distinguishes working memory, stored inputs and outputs where they differ. Recursion includes its stack. Variable sizes appear beside each bound.

CHAPTERS 0–3

First programs and debugging

Read input, split a program across files and inspect a failure in a debugger.

P001 0.7 / Walkthrough › Easy ☐

Compile a first executable

Create a minimal console program with main and a single greeting. Build it, run it, then change the greeting and rebuild so you can distinguish the old executable from the new one.

Check The second run prints the changed text once.

Time O(1) · **Space** O(1)

P002 0.13 / Guided check › Easy ☐

Verify the compiler's language mode

Compile the lesson's standard-version checker in your normal project. Change the selected language standard, rebuild and compare its reported version with your compiler settings.

Check The output agrees with the enabled mode, including the MSVC-specific reporting path when relevant.

Time Build configuration task · **Space** O(1) runtime for a fixed version table

---

P003 1.5 / Q1 › Easy ☐

Probe integer input

Read one int per run. Record its value and the stream state for a letter, a word, a decimal, a negative, an oversized number, leading whitespace and mixed letters and digits in both orders.

Check Compare 7x with x7, and 2.8 with 2.2.

Time O(b) for b input characters inspected · **Space** O(1)

P004 1.5 / Q2 › Easy ☐

Echo three values

Read three integers in sequence and print all three, in input order, in one punctuated sentence. Put a short comment above main explaining its purpose.

Check Input 3 0 -8 must preserve the zero and minus sign.

Time O(1) · **Space** O(1)

CHAPTERS 0–3

First programs and debugging

Read input, split a program across files and inspect a failure in a debugger.

P005 1.11 / In-lesson debug task › Easy ☐

Make the input stream read

A program meant to read an integer uses the output operator with std::cin. Find the first actionable diagnostic, repair the expression and confirm that entering a negative value succeeds.

Check Reading -5 places numeric -5 in the destination.

Time O(1) · **Space** O(1)

P006 1.11 / Guided task + Q1 › Easy ☐

Double and triple an input

Start with a program that doubles one integer. Extend it to print the original value alongside its double and its triple, each on its own result line.

Check Input -3 gives -6 and -9.

Time O(1) · **Space** O(1)

P007 1.x / Q3 › Easy ☐

Sum and difference

Read two integers in order. Print the sum and the difference, using the first input as the left operand.

Check Inputs 4 and 9 give 13 and -5.

Time O(1) · **Space** O(1)

P008 2.4 / Q1–2 › Easy ☐

Repair two multiplication functions

In one version a void function returns a product that main tries to print. In another, the caller supplies one operand to a two-parameter function and the function never returns its computed value. Repair both versions.

Check Both functions must produce 42 from 6 and 7.

Time O(1) · **Space** O(1)

CHAPTERS 0–3

First programs and debugging

Read input, split a program across files and inspect a failure in a debugger.

P009 2.4 / Q4–5 › Easy ☐

Build and call a doubling function

Write doubleNumber(int) as a pure calculation. Then add a main that reads an integer, calls the function and prints its return value.

Check Call it directly with 0, then run the program with -4.

Time O(1) · **Space** O(1)

P010 2.7 / Q4 › Easy ☐

Declare a function before its use

Declare doMath, which returns int and takes four named int operands. Place its definition after a caller and verify that their signatures agree.

Check The caller compiles against the declaration alone.

Time Declaration only · **Space** No runtime cost

P011 2.7 / Q5a–f › Easy ☐

Separate compilation from linkage

For the lesson's declaration/definition variants, predict which stage rejects each one. Build to confirm, then make the declarations match their definitions and ensure every called function has a definition.

Check Compile a caller with only a correct declaration, then link once with and once without the definition.

Time Build diagnostics · **Space** No runtime cost

P012 2.8 / Q1 › Easy ☐

Link a two-file program

Put the integer-reading function in input.cpp. In main.cpp declare it, call it twice and display the sum. Compile and link both translation units.

Check Input -2 and 7 gives 5 in the linked program.

Time O(1) · **Space** O(1)

CHAPTERS 0–3

First programs and debugging

Read input, split a program across files and inspect a failure in a debugger.

P013 2.12 / Q1 › Easy ☐

Protect a reusable header

Put the declaration of an integer addition function in add.h and surround it with a uniquely named preprocessor guard. Include it twice in one .cpp file.

Check Inspect preprocessed output and find one copy of the guarded declaration.

Time Preprocessing task · **Space** No runtime cost

P014 2.x / Q1–3 › Easy ☐

Give a calculator an I/O interface

Build an integer adder in main.cpp with separate reading and writing functions. Move those two functions into io.cpp, first using declarations in main.cpp, then using one guarded io.h included by both files.

Check All three builds produce 5 from 8 and -3.

Time O(1) · **Space** O(1)

P015 3.6 / Guided debugger lab › Easy ☐

Step through a function call

In a debug build, step into a small function that prints its integer argument. Repeat with step over, then use step out from inside it and note where execution resumes.

Check The debugger shows the argument inside the function and returns control to main.

Time Debugger exercise · **Space** O(1) sample program

P016 3.7 / Guided debugger lab › Easy ☐

Run to a line and set a breakpoint

In the printing example, run to the line inside the helper. Repeat with a breakpoint, resume execution and observe how the stopping location differs from a single step.

Check The value is available to inspect before the output statement executes.

Time Debugger exercise · **Space** O(1) sample program

P017 3.8 / Guided debugger lab › Easy ☐

Watch a changing value

Track an int initialized to 1 and then incremented by 2 and 3. Add it and a derived expression to the watch window and step over each assignment.

Check Record the successive values 1, 3 and 6.

Time Debugger exercise · **Space** O(1) sample program

CHAPTERS 0–3

First programs and debugging

Read input, split a program across files and inspect a failure in a debugger.

P018 3.9 / Guided lab + call-stack practice › Easy ☐

Inspect nested calls

Make main call a directly and then call b, which calls a. Break inside a on each path and compare the frames in the debugger.

Check Only the second stop has b between a and main.

Time O(d) frames inspected for depth d · **Space** O(d) call stack

P019 3.x / Q1 › Easy ☐

Find the discarded input

Step through an adder whose reader takes an integer by value and returns the newly entered number. The first call discards its result. Watch the caller's variable and fix the data flow.

Check 11 plus 4 must give 15.

Time O(1) · **Space** O(1)

P020 3.x / Q2 › Easy ☐

Find the overwritten operand

A division program reads both operands into the same destination and leaves the divisor zero. Break before division and repair the assignments.

Check For inputs 15 and 3, the divisor watch shows 3 and the result is 5.

Time O(1) · **Space** O(1)

P021 3.x / Q3 › Easy ☐

Reconstruct an active call stack

In the linked multi-function snippet, list the active functions when execution reaches line 4. Verify which invocation is active by stopping there in a debugger.

Check The most recent active call appears at the top and main is the oldest frame.

Time O(d) to list depth d · **Space** O(d) call stack

P022 3.x / Q4 › Easy ☐

Distinguish a digit from its character code

An addition helper extracts input into char and returns it as a number. Follow a digit through extraction, conversion and addition, then make the program read numeric values.

Check Inputs 2 and 8 must sum to 10.

Time O(1) · **Space** O(1)

CHAPTERS 4–6, O

Numbers, strings and operators

Work with arithmetic, text, bit flags and small functions.

P023 4.10 / Q2 › Easy ☐

Recognize prime digits

Accept a digit from 0 to 9 and report whether it is prime. Implement the decision with the conditional logic introduced in the lesson.

Check 1 is nonprime, 2 is prime and 9 is nonprime.

Time O(1) · **Space** O(1)

P024 4.10 / Q3 › Easy ☐

Simplify a height predicate

A ride admits people at least 140 cm tall. Replace the true/false branches with a single Boolean return expression, preserving equality at the boundary.

Check 139.9 fails and 140.0 passes.

Time O(1) · **Space** O(1)

P025 4.12 / Q1–2 › Easy ☐

Print a character and its code

Read one character and display both the character and its integer code. First use static_cast, then write a variation where a typed assignment or function parameter performs the conversion.

Check On an ASCII system, uppercase Z gives 90 in both versions.

Time O(1) · **Space** O(1)

P026 4.x / Q2 › Easy ☐

Evaluate a floating-point expression

Read two doubles followed by one of +, -, * or /. Print the expression and result. An unsupported operator should produce no result line. Use a nonzero divisor for the division test.

Check 5.5 and 2 with / gives 2.75. With ? there is no result line.

Time O(1) · **Space** O(1)

P027 4.x / Q3 › Intermediate ☐

Sample a falling ball for five seconds

Read a tower height in metres. From rest, report the ball's height for integer seconds 0 through 5 using g = 9.8 m/s² and distance = g × t² / 2. For every sample at or below zero, report ground.

Check From 20 m the heights at 1 s and 2 s are 15.1 m and 0.4 m.

Time O(1) for six reports · **Space** O(1)

CHAPTERS 4–6, O

Numbers, strings and operators

Work with arithmetic, text, bit flags and small functions.

P028 5.7 / Q1 › Easy ☐

Combine line input with numeric input

Read a full name, including spaces, and an age. Add the string's length to the age and print the result. Use ASCII input for the length check.

Check Ada Lin has 7 characters including the space. At age 30 the result is 37.

Time O(n) to read n characters · **Space** O(n) for the name

P029 5.x / Q2 › Easy ☐

Repair an age check

A program interprets typed ages as characters, embeds the threshold 16 directly in its condition and prints a multi-character literal for punctuation. Correct all three defects.

Check Test ages 9, 15 and 16, including the line ending.

Time O(1) · **Space** O(1)

P030 5.x / Q4 › Easy ☐

Compare two named people

Read a full name and an integer age for each of two people, then identify the older person and include both names and ages in the comparison. Use distinct ages for the source case.

Check A multiword second name must survive the preceding numeric input.

Time O(n) for total name length n · **Space** O(n) for names

P031 6.1 / Q1a–d › Easy ☐

Make expression grouping explicit

Parenthesize the four lesson expressions so arithmetic, chained assignment, prefix increment and mixed logical operators keep their original grouping. Leave the operators and operands unchanged.

Check Check that chained assignment groups rightward and that logical AND binds more tightly than OR.

Time Expression rewrite · **Space** O(1) per expression

P032 6.3 / Q2 › Easy ☐

Test integer parity

Implement constexpr isEven(int), then read a number and report its parity using the function.

Check Check -3, -2 and 0.

Time O(1) · **Space** O(1)

CHAPTERS 4–6, O

Numbers, strings and operators

Work with arithmetic, text, bit flags and small functions.

P033 6.8 / Q2 › Easy ☐

Express parity with logical negation

Rewrite the remainder-based parity predicate using logical negation in place of an equality comparison. Keep the function correct for negative inputs.

Check -8 and 0 are even, and -7 is odd.

Time O(1) · **Space** O(1)

P034 6.x / Quiz › Easy ☐

Describe an apple count in words

Map counts below 0, 0, 1, 2, 3 and above 3 to “negative”, “no”, “a single”, “a couple of”, “a few” and “many”, respectively. Write a second helper that uses the conditional operator to choose the singular or plural noun.

Check Check that 1 gives “a single apple” and 3 gives “a few apples”.

Time O(1) · **Space** O(1)

P035 O.2 / Q2–3 › Intermediate ☐

Rotate a four-bit value

Rotate a std::bitset<4> left by one position with wraparound. Build one version with test/set and a second using bitwise operators alone.

Check 1001 becomes 0011. Four applications restore every input.

Time O(1) for four bits · **Space** O(1)

P036 O.3 / Q1a–c › Easy ☐

Edit flags without disturbing neighbours

A byte encodes article flags. Using integer masks, mark an article viewed, test its deleted flag, then clear the favourite flag. Assign masks 0x01, 0x10 and 0x04 respectively.

Check Starting at 0x04 gives 0x05 after marking viewed, false for deleted, then 0x01.

Time O(1) · **Space** O(1)

P037 O.4 / Q6 › Intermediate ☐

Format an unsigned byte

Read an integer in [0,255] and print exactly eight binary digits with a space between the nibbles. Use arithmetic and comparisons without bitwise operators or std::bitset.

Check 9 gives 0000 1001, and 255 gives 1111 1111.

Time O(1) for eight bits · **Space** O(1)

CHAPTERS 7–9

Control flow and validation

Make branching, loops, random choices and error paths observable.

P038 7.3 / Q1 › Easy ☐

Put two input values in order

Read two integers into smaller and larger. When they arrive in reverse order, swap them using a temporary whose scope ends with the conditional block. Report their final values.

Check 6 and -2 must finish as -2 and 6.

Time O(1) · **Space** O(1)

P039 7.x / Q1 › Easy ☐

Fix a conditional block

A missing pair of braces makes a sign change run for positive inputs too. Restrict the message and negation to the negative branch. Use representable positive magnitudes.

Check Both -8 and 8 finish as 8. Only the negative input triggers the message.

Time O(1) · **Space** O(1)

P040 7.x / Q2 › Easy ☐

Publish a class-size constant

In constants.h expose constants::maxClassSize with value 35. Use the C++17 inline constexpr form and include your header from the supplied client.

Check Build two translation units that include the header.

Time Declaration only · **Space** No per-use storage allocation

P041 7.x / Q3 › Easy ☐

Maintain a running total

Implement accumulate(int) to add its argument to the total from all earlier calls in this process. Keep the state inside the function.

Check A fresh run with 4, -1, 6 returns 4, 3, 9.

Time O(1) per call · **Space** O(1) persistent

CHAPTERS 7–9

Control flow and validation

Make branching, loops, random choices and error paths observable.

P042 8.6 / Q1 › Easy ☐

Dispatch integer arithmetic

Accept two integers and an operator character. Use switch to implement addition, subtraction, multiplication, integer division and remainder. Report an unknown operator. The source assumes valid divisors.

Check 11 / 4 yields 2 and 11 % 4 yields 3.

Time O(1) · **Space** O(1)

P043 8.8 / In-lesson debug task › Easy ☐

Stop an unsigned countdown

Diagnose a while loop that counts down through zero with an unsigned counter and then wraps. Preserve the countdown and the final blastoff message while ensuring termination.

Check Set the start to 2 and verify that control returns after one blastoff.

Time O(n) for start n · **Space** O(1)

P044 8.8 / Q2 › Easy ☐

Print the alphabet and its codes

Iterate a char variable over lowercase ASCII letters, printing the character and its numeric code on each line.

Check The first and last rows pair a with 97 and z with 122.

Time O(1) for 26 rows · **Space** O(1)

P045 8.8 / Q3–4 › Intermediate ☐

Draw two descending-number triangles

Use nested while loops. First print rows of lengths 5 down to 1, each counting down to 1. Then print lengths 1 up to 5, right-aligned with the final 1 in the same column.

Check With three rows the second pattern ends in the row 3 2 1.

Time O(r^2) cells for r rows · **Space** O(1) streamed

CHAPTERS 7–9

Control flow and validation

Make branching, loops, random choices and error paths observable.

P046 8.10 / Q1–3 › Easy ☐

Practice counted loops

Write three short loops: emit the even integers from 0 through 20, accumulate 1 through n, and repair a descending 9-to-0 loop whose unsigned condition never becomes false.

Check sumTo(6) gives 21. The countdown prints ten values and terminates.

Time O(n) for the sum, O(1) for fixed ranges · **Space** O(1)

P047 8.10 / Q4–5 › Intermediate ☐

Extend FizzBuzz

For each integer from 1 through a limit, substitute fizz for multiples of 3 and buzz for multiples of 5. Use one if/else chain first. Add pop for 7, combine matches in divisor order and use just four if statements for the 150-number version.

Check 15 gives fizzbuzz, 35 gives buzzpop and 105 gives fizzbuzzpop.

Time O(n) fixed-width steps · **Space** O(1) streamed

P048 8.14 / In-lesson experiment › Easy ☐

Reproduce a repeated dice sequence

Run the lesson's default-seeded dice program three times. Explain the repeated sequence, then change the seeding strategy while keeping a fixed seed available for repeatable tests.

Check With the same seed, engine and distribution implementation, reproduce the first run.

Time Expected O(r) for r fixed-width rolls · **Space** O(1) fixed engine state

P049 8.x / Q1 › Intermediate ☐

Simulate until the ball lands

Extend the tower program from 4.x. Produce one report per whole second starting at zero and stop immediately after the first report at ground. Preserve fractional heights.

Check A 20 m tower produces reports at 0, 1, 2 and 3 seconds.

Time O(t) reports to impact · **Space** O(1)

CHAPTERS 7–9

Control flow and validation

Make branching, loops, random choices and error paths observable.

P050 8.x / Q2 + extra › Intermediate ☐

Test larger integers for primality

Write a loop-based divisor test. Extend it to skip unnecessary even divisors and stop once a factor pair would repeat. Handle all integers below 2.

Check Try -1, 2, 49 and 97. The square 49 exercises the loop boundary.

Time Baseline O(n), optimized O(√n) · **Space** O(1)

P051 8.x / Q3 + extra › Intermediate ☐

Build the Hi-Lo guessing game

Choose a random integer from 1 to 100. Allow seven guesses, indicate higher or lower after each miss and reveal the target on loss. Handle replay answers and make bounds and guess count configurable.

Check Win on the last attempt, lose once, then replay with a fresh secret.

Time O(g) for g guesses per round · **Space** O(1)

P052 9.2 / Q2 › Easy ☐

Design branch-coverage tests

For a lowercase-vowel predicate with a switch, construct tests for ordinary vowels, consonants and y with both settings of its Boolean option. Identify which returns and cases execute.

Check Include the default case and both possible outcomes for y.

Time O(1) per test · **Space** O(1)

P053 9.x / Q1 › Intermediate ☐

Make Hi-Lo input recoverable

Update the guessing game to handle failed extraction, a guess outside the configured interval, trailing characters and extra input at the replay prompt. Keep later reads synchronized.

Check Feed a letter, 101 and 43x on separate lines. Verify the next valid line is read cleanly.

Time O(b) for b input characters processed · **Space** O(1) with streamed discard

CHAPTERS 10–13, F

Types, functions and pointers

Express units, deduction, aliases, lifetime and data representation.

P054 10.7 / Q1 › Easy ☐

Name an error result with a type alias

Define PrintError as an alias for int. Change the declared return type of printData() to use the alias and check that callers remain compatible.

Check Assign the function result to an int in a compiling client.

Time Declaration only · **Space** No runtime cost

P055 10.x / Q2a › Easy ☐

Label the units of an angle conversion

Create distinct alias names for values measured in degrees and radians. Apply them to the parameter, return type and relevant variables in a degrees-to-radians calculator.

Check 180 degrees produces approximately 3.14159 radians.

Time O(1) · **Space** O(1)

P056 11.9 / Q1 › Intermediate ☐

Compute a factorial at compile time

Write a constexpr factorial template with an integer non-type template parameter. Handle zero and make a negative argument ill-formed. Use test values whose factorial fits your return type.

Check static_assert(factorial<5>() == 120) passes. A negative specialization must fail to build.

Time O(n) multiplies during evaluation · **Space** O(1) iterative workspace

P057 11.10 / In-lesson linkage problem › Intermediate ☐

Make a template usable across files

Reproduce a caller that sees an addOne template declaration while its definition is in another .cpp. Fix visibility through a header and try the explicit-instantiation approach for a fixed set of types.

Check Verify that both an int call and a double call link with the header approach.

Time Build/link task · **Space** O(1) for built-in addition

P058 11.x / Q2, steps 1–3 › Intermediate ☐

Generalize three arithmetic operations

Write add for matching types, mult for a value and an int, and sub for two potentially different types. Return the first operand type from mult and deduce the subtraction result.

Check Add two doubles, multiply 2.5 by 4 and subtract 0.25 from 3.

Time O(1) for built-in arithmetic · **Space** O(1)

CHAPTERS 10–13, F

Types, functions and pointers

Express units, deduction, aliases, lifetime and data representation.

P059 F.X / Q1 › Intermediate ☐

Make the tower calculation const-correct

Revisit the fixed-sample tower program. Separate user input and printing from calculations that can be constexpr. Mark values const or constexpr according to how their initializers obtain values.

Check A constant argument can exercise the pure calculation at compile time. A typed height still works at runtime.

Time O(1) for the fixed schedule · **Space** O(1)

P060 12.7 / Q2 › Easy ☐

Reseat a pointer correctly

A pointer already points at one int. Correct the expression that tries to assign a second address through *ptr when the intent is to point at the second int.

Check After reseating, changing *ptr changes only the second integer.

Time O(1) · **Space** O(1)

P061 12.x / Q5 › Easy ☐

Order two integers in place

Implement sort2(int&, int&) so the first caller variable holds the smaller value on return. Do nothing when the values are already ordered.

Check Try a reversed pair, equal values and two references to the same int.

Time O(1) · **Space** O(1)

P062 13.2 / Q1–2 › Easy ☐

Name monster kinds with an enum

Define unscoped enumerators for orc, goblin, troll, ogre and skeleton. Move the enum into a namespace, then create a troll from main using the qualified name.

Check The program compiles without introducing those enumerators into global scope.

Time O(1) · **Space** O(1)

P063 13.6 / Q1 › Easy ☐

Describe an animal through a scoped enum

Use an enum class for pig, chicken, goat, cat, dog and duck. Implement switches that supply an animal name and report its leg count.

Check Cat reports four legs and chicken reports two.

Time O(1) for six species · **Space** O(1)

CHAPTERS 10–13, F

Types, functions and pointers

Express units, deduction, aliases, lifetime and data representation.

P064 13.10 / Q1 › Easy ☐

Compute advertising revenue

Store ad views, click-through percentage and earnings per click in a struct. Pass it to a function that prints both the data and the income for the day.

Check 500 views, 4% clicks and 0.50 per click give 10.00.

Time O(1) · **Space** O(1)

P065 13.10 / Q2 › Easy ☐

Multiply two fraction records

Store a numerator and denominator as integers. Read two valid fractions, multiply them into a new record and print the numerator and denominator. Reduction is optional here.

Check 3/5 times 2/7 gives 6/35.

Time O(1) · **Space** O(1)

P066 13.x / Q1 › Easy ☐

Print monster records

Model ogre, dragon, orc, giant spider and slime with an enum. Combine species, owned name and health in a struct. Aggregate-initialize an ogre and a slime and print both.

Check Changing the slime's health must leave the ogre's report unchanged.

Time O(n) for n name characters written · **Space** O(1) beyond stored names

P067 13.x / Q3 › Intermediate ☐

Build a homogeneous triple

Define a Triad<T> aggregate with three values of one type. Write a print function template accepting it without copying, and support deduction for the lesson's declarations.

Check Print an integer triple and a const double triple. For C++17, supply a deduction guide if needed.

Time O(1) for three scalar fields · **Space** O(1)

P068 13.y / Q2 › Easy ☐

Replace a word in place

Given a sentence containing “red”, use a single string replacement call to change the word to “blue”. Keep the rest of the sentence and avoid assigning a new full sentence.

Check The source sentence ends in “car yesterday.” before and after replacement.

Time O(n) character movement on growth · **Space** Up to O(n) if reallocated

CHAPTERS 14–15

Classes and object design

Give related data a clear interface and check its behaviour.

P069 14.1 / Q1–2 › Easy ☐

Represent a snake two ways

Add a snake to the procedural animal program, then to its object-oriented version. In both variants instantiate the snake and print its name and number of legs.

Check Both versions identify a snake with zero legs.

Time O(1) · **Space** O(1)

P070 14.3 / Q1–2 › Easy ☐

Compare two integer pairs

Define IntPair with two integer members and a print member. Add a member that compares both coordinates against another pair.

Check (4,1) equals (4,1), but differs from (1,4).

Time O(1) · **Space** O(1)

P071 14.5 / Q2a–b › Easy ☐

Encapsulate three coordinates

Give Point3d three private integers and public operations to set them, print them and compare two points for equality.

Check Mutating only z changes equality.

Time O(1) · **Space** O(1)

P072 14.10 / Q1 › Easy ☐

Construct and print a ball

Store an owned colour string and a double radius in private data members. Use a member initializer list and provide the public access needed by a nonmember print function.

Check A ball built from a temporary colour string still prints correctly later.

Time O(c) to copy or print c colour characters · **Space** O(c) owned

CHAPTERS 14–15

Classes and object design

Give related data a clear interface and check its behaviour.

P073 14.12 / Q1–2 › Intermediate ☐

Consolidate ball constructors

Support four construction forms: no arguments, colour only, radius only and both. Use black and 10.0 as defaults. First express four overloads, then consolidate them using delegation and default arguments.

Check A radius-only call keeps black, while a colour-only call keeps 10.0.

Time O(c) to own c colour characters · **Space** O(c)

P074 14.x / Q1a–b › Intermediate ☐

Measure between 2D points

Write Point2d with double coordinates, zero defaults, a constructor and a print member. Add a const distanceTo operation that computes Euclidean distance to another point.

Check (1,1) and (4,5) are 5 units apart. A point's distance to itself is 0.

Time O(1) · **Space** O(1)

P075 14.x / Q2 › Intermediate ☐

Turn a fraction record into a class

Rebuild the earlier fraction exercise with private numerator and denominator. Convert input, multiplication and printing into instance methods and preserve the existing result.

Check Read 2/3 and 3/7. The unreduced product is 6/21.

Time O(1) · **Space** O(1)

P076 15.5 / Q1 › Intermediate ☐

Store three independent types

Build a class template with three independent type parameters. Add a constructor, const accessors and a const print member defined outside the class body.

Check Use one const object containing an integer, a double and a character.

Time O(1) for three scalar fields · **Space** O(1)

CHAPTERS 14–15

Classes and object design

Give related data a clear interface and check its behaviour.

P077 15.7 / Q1 › Intermediate ☐

Move a random generator into a class

Convert the lesson's Random namespace into a class. Keep one shared Mersenne Twister, its seeding routine and the public integer-range operation as appropriate static members.

Check Two calls compile without constructing a Random object and stay within the requested inclusive interval.

Time Expected O(1) per fixed-width draw · **Space** O(1) fixed engine state

P078 15.9 / Q1, steps 1–2 › Intermediate ☐

Translate a point using friendship

Implement Point3d::moveByVector using the three private vector components. First grant friendship to the full point class, then restrict it to that one member function.

Check (2,0,-1) moved by (-3,5,2) becomes (-1,5,1) in both versions.

Time O(1) · **Space** O(1)

P079 15.x / Q1a–e › Intermediate ☐

Build the monster value type

Define dragon, goblin, ogre, orc, skeleton, troll, vampire and zombie, plus a count sentinel. Move the enum into Monster, add a constructor for kind, name, roar and hit points, then implement type-to-text and printing.

Check A manually constructed skeleton reports its own name, sound and hit points.

Time O(c) for total text length · **Space** O(c) owned

P080 15.x / Q1f–h › Intermediate ☐

Generate random monsters

Start with a MonsterGenerator namespace returning a known monster. Add six names and six roars, then randomize kind, name, roar and health from 1 through 100.

Check With a reproducible seed, repeated generation stays within the defined tables and health range.

Time Expected O(1) selection plus O(c) string copying · **Space** O(c) result

CHAPTERS 16–19

Containers and algorithms

Choose storage, manage bounds and measure how operations scale.

P081 16.2 / Q1, Q3–4 › Easy ☐

Choose vector element types and sizes

Create a deduced vector of the first five positive squares and a 365-element floating-point temperature vector. In a separate program read three integers into a vector and print their sum and product.

Check The temperature vector has size 365. For 2, 3, 4 the sum is 9 and product 24.

Time O(n) to initialize n elements · **Space** O(n) stored

P082 16.3 / Q1 › Easy ☐

Inspect a character vector

Build a five-element vector spelling hello. Print its size, then retrieve index 1 using both unchecked subscripting and at().

Check Both accesses yield e and the size is 5.

Time O(1) · **Space** O(1)

P083 16.4 / Q1 › Easy ☐

Guard indexed access

Write a function that accepts a vector of any supported element type and an index. Print the element if the index is valid, otherwise print a diagnostic.

Check For a two-element vector, index 1 succeeds and index 2 fails.

Time O(1) lookup for scalar elements · **Space** O(1)

P084 16.6 / Q1–2 › Easy ☐

Print a vector through a reusable loop

Print [4,6,7,3,8,2,1,9] in input order with spaces and a final newline, then move the loop into a reusable function that accepts the vector by const reference.

Check A two-item vector prints the two values in their original order.

Time O(n) for n elements · **Space** O(1)

P085 16.6 / Q3–4 › Intermediate ☐

Search a validated numeric input

Read a value in [1,9], retry invalid inputs and discard trailing text after a valid numeric prefix. Print the array and report the matching index or a miss. Generalize the search to non-integer vectors.

Check In [4,7,2], 7 is at index 1 and 5 is absent. Try an exact stored decimal next.

Time O(n+b) for n elements and b input characters · **Space** O(1) streamed

CHAPTERS 16–19

Containers and algorithms

Choose storage, manage bounds and measure how operations scale.

P086 16.6 / Q5 › Easy ☐

Return the largest vector element

Write a template returning the largest value in a vector, or a value-initialized element if the vector is empty.

Check A vector of -8 and -3 returns -3. An empty int vector returns 0.

Time O(n) comparisons · **Space** O(1) for scalars

P087 16.6 / Q6 › Intermediate ☐

Drive FizzBuzz from lookup tables

Associate divisors 3, 5, 7, 11, 13, 17 and 19 with fizz, buzz, pop, bang, jazz, pow and boom. Store parallel vectors, check their lengths and emit matching words in order for 1 through 150.

Check 33 gives fizzbang. A number with no matching divisor prints numerically.

Time O(nk) for n numbers and k rules · **Space** O(k) rules

P088 16.7 / In-lesson index challenge › Intermediate ☐

Traverse a vector backwards safely

Repair a reverse loop whose unsigned index begins at size()-1 and tests index >= 0. Preserve reverse order and make empty vectors safe.

Check [] produces no values. [5,8] produces 8 then 5.

Time O(n) for n elements · **Space** O(1)

P089 16.8 / Q1–2 › Easy ☐

Search a list with a range-for

Look up an entered name among Alex, Betty, Caroline, Dave, Emily, Fred, Greg and Holly using a range-based loop. Extract a generic Boolean membership function that works for other vector types.

Check Test the first and last names, a missing name and an integer vector.

Time O(n) comparisons, up to O(nL) for L-character names · **Space** O(1) search

P090 16.9 / Q1 › Easy ☐

Look up leg counts by enum

Use an unscoped enum inside a namespace to index leg counts for chicken, dog, cat, elephant, duck and snake. Assert that the table size matches the number of animals.

Check Lookup by the elephant enumerator gives 4.

Time O(1) lookup · **Space** O(k) for k table entries

CHAPTERS 16–19

Containers and algorithms

Choose storage, manage bounds and measure how operations scale.

P091 16.11 / Q1 › Easy ☐

Print stack changes

Use a vector as a stack. Display its contents after pushing 1, 2, 3, popping once, pushing 4 and popping until empty. Label empty states explicitly.

Check The pop after pushing 4 leaves 1 and 2, in that order.

Time Push amortized O(1), pop O(1), display O(k) at depth k · **Space** O(k) stored

P092 16.x / Q1a–e › Easy ☐

Distinguish vector initialization forms

Define vectors for six even numbers, constant doubles, four constant name views, one int whose value is 12, and twelve value-initialized ints. Use deduction where it gives the intended type.

Check The final two vector sizes are 1 and 12.

Time O(n) to initialize n slots · **Space** O(n) stored

P093 16.x / Q2 › Intermediate ☐

Report a game inventory

Use an enum to index counts for health potions, torches and arrows. Start with 1, 5 and 10. Verify the table's size, pluralize each label and report the total.

Check The total is 16. Only the health potion takes its singular label.

Time O(k) for k item types · **Space** O(k) stored

P094 16.x / Q3–4 › Intermediate ☐

Return extrema and their positions

Return the minimum and maximum indices for a nonempty vector of numbers. Extend the program to collect integers until -1, show their values and positions, and handle an immediate sentinel.

Check In [4,1,9,9], the first maximum is at index 2.

Time O(n) time · **Space** O(1) scan, O(n) collected input

CHAPTERS 16–19

Containers and algorithms

Choose storage, manage bounds and measure how operations scale.

P095 16.x / Q5, steps 1–4 › Advanced ☐

Build C++man

Pick a secret word, mask unrevealed letters and accept one letter per turn. Reveal all matches, ignore repeats, alphabetize wrong guesses and display the remaining miss allowance. End on a solved word or exhausted allowance.

Check With secret level, a single e reveals both occurrences. Repeating e costs no turn.

Time O(gw) for g guesses and w letters, fixed alphabet · **Space** O(w) state

P096 17.1 / Q2–3 › Easy ☐

Initialize fixed-size arrays

Define a std::array for 365 daily high temperatures to tenths of a degree. Initialize another std::array with the characters of hello and print its second element.

Check Temperature storage has 365 slots. Index 1 in the second array gives e.

Time O(n) if n slots are initialized · **Space** O(n) stored

P097 17.2 / Q1 › Easy ☐

Compare three array access forms

Print the size of a five-character std::array, then read its second character with operator[], at() and std::get<1>().

Check All three produce e for hello.

Time O(1) · **Space** O(1)

P098 17.3 / Q1 › Intermediate ☐

Print arrays of different lengths and types

Write one function template that accepts a const std::array with element type and length inferred. Print its contents comma-separated in parentheses, then report the length.

Check For [2,5], the list is (2, 5) and length is 2. Test a char array too.

Time O(n) for n scalar elements · **Space** O(1)

CHAPTERS 16–19

Containers and algorithms

Choose storage, manage bounds and measure how operations scale.

P099 17.4 / Q1–2 › Easy ☐

Initialize an array of item records

Define Item with a name view and gold value. Build a four-item std::array using class template argument deduction, then build it again without repeating the Item type at each element.

Check Print names and prices for sword (5), dagger (3), club (2) and spear (7) from either array.

Time O(n) records · **Space** O(n) stored

P100 17.6 / Quiz / Q1 › Intermediate ☐

Build an animal data directory

Map chicken, dog, cat, elephant, duck and snake to records containing name, legs and sound. Read a name and print its record followed by the remaining animals, or a diagnostic followed by the whole table if unmatched.

Check A known animal appears once overall. An unknown name leaves all six in the remaining list.

Time O(kL) for k names of maximum length L · **Space** O(k) records plus input

P101 17.7 / Q1–2 › Easy ☐

Repair built-in array declarations

Declare a zero-initialized constexpr C-style array of three ints. Fix a second program whose array bound is a runtime variable, whose final access is one past the end and which writes to a const array.

Check A strictly conforming build accepts the declaration. Every access stays within bounds.

Time O(n) when initializing n elements · **Space** O(n) stored

P102 17.7 / Q3 › Easy ☐

Recognize single-digit squares

Store the perfect squares in [0,9] in a global constexpr C-style array. Read digits and report membership until the user enters -1.

Check 0, 4 and 9 succeed, 8 fails and -1 exits.

Time O(g) for g guesses against a fixed table · **Space** O(1)

CHAPTERS 16–19

Containers and algorithms

Choose storage, manage bounds and measure how operations scale.

P103 17.10 / Q1–2 › Intermediate ☐

Traverse a C string in both directions

Print a null-terminated string one character at a time using pointer arithmetic. Repeat in reverse order. Support the empty string without forming a pointer before its beginning.

Check For “C++” the reverse output is “++C”.

Time O(n) for n characters · **Space** O(1)

P104 17.x / Q1a–c › Intermediate ☐

Fix three array bugs

Repair a loop that includes index size(), a range-for over a decayed C-array parameter and an std::array whose extent is read at runtime. Preserve each program's intended traversal.

Check Exercise empty runtime input and a one-element traversal.

Time O(n) traversal · **Space** O(1) scan, O(n) for runtime storage

P105 17.x / Q2, steps 1–3 › Advanced ☐

Run a potion shop

Stock healing, mana, speed and invisibility potions at 20, 30, 12 and 50 gold. Give a named player 80–120 gold, accept purchases until quit, reject invalid or unaffordable selections and print inventory and balance.

Check With 80 gold, buy speed twice. The final count is 2 and the balance is 56.

Time O(gk) for g menu visits and k potion types · **Space** O(k) inventory

P106 17.x / Q3, steps 1–4 › Intermediate ☐

Construct, shuffle and deal a deck

Represent 13 ranks and four suits, give cards compact display codes and blackjack values, then construct all 52 distinct pairs. Shuffle resets the dealing cursor. Guard exhaustion.

Check Deal all cards once and verify 52 distinct pairs.

Time O(d) build/shuffle, O(1) per deal · **Space** O(d) deck

CHAPTERS 16–19

Containers and algorithms

Choose storage, manage bounds and measure how operations scale.

P107 17.x / Q4, Q5c › Advanced ☐

Play blackjack and upgrade scoring

Deal one dealer card and two player cards. Allow hit or stand. A score above 21 busts, and the dealer draws below 17. Initially aces count as 11 and ties favour the dealer. Then add flexible aces, visible opening cards and a distinct tie result.

Check Ace, ace, nine can total 21. Equal unbusted final scores draw in the upgraded game.

Time O(d) shuffle and O(h) for h draws · **Space** O(d) deck, O(1) scoring

P108 18.1 / Q1–2 › Intermediate ☐

Trace and reverse selection sort

Trace [30,60,20,50,40,10] after each selection-sort swap. Change the implementation to place the largest remaining value at the front of each unsorted portion.

Check Try [4,2,4,1]. Descending output is [4,4,2,1].

Time O(n^2) comparisons, O(n) swaps · **Space** O(1)

P109 18.1 / Q3–4 › Intermediate ☐

Implement and optimize bubble sort

Compare adjacent values and swap out-of-order neighbours. Skip the settled suffix on each pass and stop after a pass with no swaps. When a pass ends the sort early, print its one-based pass number.

Check A sorted input requires one checking pass. Reverse order exercises the worst case.

Time Worst O(n^2), best O(n) after optimization · **Space** O(1)

P110 18.4 / Guided timing lab › Intermediate ☐

Compare two sorting implementations

Use the lesson's chrono-based timer to compare selection sort and std::sort. Give both the same unsorted input and keep printing outside the timed section.

Check Verify equal sorted outputs before comparing durations at two input sizes.

Time Selection O(n^2), std::sort O(n log n) comparisons · **Space** O(n) for an input copy

P111 19.2 / Q1 › Intermediate ☐

Sort a runtime-sized name list

Read a count, dynamically allocate an array of strings, read full names, sort them and print a numbered list. Release the array, including for an empty list.

Check Try two names with spaces, a repeated name and a count of zero.

Time O(n log n) comparisons, up to O(L) each · **Space** O(n+C) for n names with C total characters

CHAPTERS 20–21

Functions and operators

Practice recursion, callable objects and value-like class interfaces.

P112 20.1 / Q1a–e › Intermediate ☐

Select arithmetic by callable

Read two ints and validate an operator from +, -, * and /. Implement four arithmetic functions. Define the callable type, select a function by operator and invoke it through the returned callable.

Check 14 / 3 uses integer division and returns 4. An invalid operator triggers another prompt.

Time O(a) for a input attempts, O(1) dispatch · **Space** O(1)

P113 20.3 / In-lesson extension › Intermediate ☐

Calculate Fibonacci iteratively

Replace the branching recursive Fibonacci example with an iterative computation for the same nonnegative index. Keep only the state needed to advance one term.

Check With F(0)=0 and F(1)=1, check F(10)=55.

Time O(n) time · **Space** O(1) state

P114 20.3 / Q1 › Intermediate ☐

Compute factorial recursively

Return n! for a nonnegative integer using a base case and one recursive call per smaller value. Print the first seven factorials starting at zero.

Check 0! is 1 and 6! is 720. Limit tests to representable results.

Time O(n) calls · **Space** O(n) call stack

P115 20.3 / Q2 › Intermediate ☐

Add decimal digits recursively

For a positive integer, peel off one decimal digit at each recursive step and return the sum.

Check 8042 gives 14. An internal zero contributes zero.

Time O(d) for d digits · **Space** O(d) call stack

P116 20.3 / Q3a–b › Intermediate ☐

Print binary with recursion

Print a positive integer in binary using recursive division by two. Extend the routine to zero and the unsigned bit representation of negative inputs at the chosen integer width.

Check 10 gives 1010, zero gives 0 and -1 gives all ones for the chosen width.

Time O(b) for b printed bits · **Space** O(b) call stack

CHAPTERS 20–21

Functions and operators

Practice recursion, callable objects and value-like class interfaces.

P117 20.6 / Q1–2 › Intermediate ☐

Use lambdas with standard algorithms

Find the highest-scoring student with max_element and a comparison lambda. In a separate array sort seasons by increasing mean temperature with std::sort and a lambda.

Check Scores 6, 9, 3 select the middle record. Negative temperatures sort before positive ones.

Time O(n) maximum, O(n log n) sort comparisons · **Space** O(1) scan, sort workspace implementation-dependent

P118 20.7 / Q3 › Advanced ☐

Guess the scaled squares

From a starting integer generate n consecutive squares and multiply each by the same random value in [2,4]. Remove each correct guess. A miss ends play and reports the closest remaining value.

Check Starting at 2 with multiplier 3 yields 12, 27 and 48 for n=3.

Time O(n^2) worst for searches and vector erasures over a game · **Space** O(n)

P119 20.x / Q1a–c › Easy ☐

Specify three function interfaces

Declare a maximum of two doubles, a swap that mutates two ints, and a largest-element lookup on a nonempty dynamic int array that returns a modifiable reference. Include the array length.

Check Assign through the returned reference in a client expression.

Time Declarations only · **Space** No runtime cost

P120 20.x / Q2a–e › Intermediate ☐

Diagnose five function failures

Review a dangling reference to a local array, unbounded recursion, overloads differing only by return type, an oversized automatic array and unsafe command-line indexing and string-to-integer conversion. Repair each with its intent intact.

Check Test the recursion at its base and invoke the command-line program with no extra arguments.

Time O(1) access or O(n) traversal/summation · **Space** O(1) to O(n)

P121 20.x / Q3a–b › Intermediate ☐

Implement binary search twice

Search a sorted integer array by narrowing its index interval. Return the matching index or -1. Write iterative and recursive versions with the same contract.

Check Both handle an empty range, a missing key and hits at either end.

Time O(log n) time · **Space** O(1) iterative, O(log n) recursive stack

CHAPTERS 20–21

Functions and operators

Practice recursion, callable objects and value-like class interfaces.

P122 21.2 / Q1a–b, e › Intermediate ☐

Multiply and reduce fractions

Encapsulate integer numerator and denominator. Print the fraction and overload multiplication as friend functions for two fractions and for an int on either side. Reduce results using gcd.

Check 6/8 times 2/3 reduces to 1/2. Both operand orders work for integer multiplication.

Time O(log m) arithmetic steps for gcd, m = larger magnitude · **Space** O(1) iterative workspace

P123 21.4 / Q1 › Intermediate ☐

Stream a fraction atomically

Add insertion and extraction operators for numerator/denominator text. Validate both numbers, the slash and a nonzero denominator before committing. Preserve the old fraction and set stream failure on bad input.

Check Reading 7/0 into 2/5 must leave 2/5 intact and signal failure.

Time O(b) for b input/output characters · **Space** O(1) numeric temporaries

P124 21.6 / Q1 › Easy ☐

Implement unary plus on a point

Add a const unary + to a three-dimensional point. Return a point with the same signed coordinates and independent value semantics.

Check Applying + to (-2,0,3) preserves -2.

Time O(1) · **Space** O(1)

P125 21.7 / Q1 › Intermediate ☐

Compare rational numbers

Implement all six comparisons for Fraction. Make equivalent reduced values compare equal and use a consistent numeric order. State your sign convention and test only products representable by your comparison type.

Check 1/2 equals 2/4 and is less than 3/4.

Time O(1) per comparison for fixed-width normalized fractions · **Space** O(1)

P126 21.7 / Q2 › Intermediate ☐

Order and print cars

Give Car a stream representation and an ordering by make, then by model for equal makes. Use std::sort on a heterogeneous list of makes and models.

Check Two cars of the same make sort by model even when supplied in reverse order.

Time O(n log n) comparisons, up to O(L) per string comparison · **Space** Sort workspace implementation-dependent

CHAPTERS 20–21

Functions and operators

Practice recursion, callable objects and value-like class interfaces.

P127 21.9 / Q1a–c, Q2–3 › Intermediate ☐

Build and review a grade map

Store student names and letter grades in a vector of records. Implement subscript lookup returning a grade reference and insert a default record on a miss. Examine the cost and the lifetime of references across later insertions.

Check Assign a grade through the subscript, insert another student and look up the first student again by name.

Time O(nL) lookup for n names up to L characters · **Space** O(n+C) records and C name characters

P128 21.10 / Q1–2 › Intermediate ☐

Slice an owned string, then borrow it

Give MyString stream output and a call operator (start, length). First return an owned substring. Then return a string_view into the owner's character storage and document when it is valid.

Check For “mesh faces”, (5,5) selects “faces”. Keep the owner alive for the view test.

Time O(k) time/space to copy k characters, O(1) time/space for a view · **Space** O(1) additional beyond result

P129 21.x / Q2a › Intermediate ☐

Stream a running average

Track an integer sum and count. Overload += so updates can be chained and << so a nonempty accumulator displays its floating-point mean.

Check Adding 2 and 5 gives 3.5. Check that a copy can then evolve independently.

Time O(1) per update or query · **Space** O(1)

P130 21.x / Q3 › Advanced ☐

Own and copy an integer array

Build a heap-backed IntArray with checked preconditions, indexing, printing, destruction, deep copying and assignment. Accept a requested size without std::array or std::vector.

Check Assign across different sizes, self-assign and modify the copy. The original must retain its elements.

Time O(n) copy or print, O(1) subscript · **Space** O(n) owned or newly copied

P131 21.x / Q4, steps 1–5 › Advanced ☐

Represent signed hundredths

Implement FixedPoint2 with whole and hundredth parts. Normalize signs and carry excess hundredths. Add rounded construction from double, conversion back, streaming, equality, unary negation and addition.

Check (2,-8) represents -2.08. Adding 0.75 and 0.50 gives 1.25.

Time O(1) arithmetic on fixed-width values · **Space** O(1)

CHAPTER 21.Y

The 15-puzzle

Take the eight official project stages from board representation to a playable puzzle.

P132 21.y / Project, steps 1–3 › Intermediate ☐

Design and draw the 15-puzzle

Decompose a sliding puzzle into responsibilities. Create a printable Tile, an empty tile and a 4×4 Board that starts solved. Lay out the numbers 1–15 and the blank in reading order.

Check Compare row widths for single- and double-digit tiles. The blank must occupy one full cell.

Time O(c) to initialize or display c cells · **Space** O(c) board

P133 21.y / Project, steps 4–6 › Intermediate ☐

Interpret sliding commands

Accept w, a, s and d for tile movement and q to quit. Map movements to a Direction with an opposite and a random generator. Use a comparable Point to compute adjacent coordinates.

Check Moving left then right returns the starting point. Invalid input leaves the board alone.

Time O(b) for b input characters, O(1) per direction · **Space** O(1)

P134 21.y / Project, steps 7–8 › Advanced ☐

Move tiles and detect a solved puzzle

Slide the neighbour opposite the requested tile direction into the blank when it is on the board. Scramble the solved board through legal moves, redraw accepted moves and stop when the original arrangement returns.

Check Reverse a recorded legal scramble. Every tile must return to its original position.

Time O(c) per scan/redraw, O(sc) for s scramble attempts with a scan · **Space** O(c) board

CHAPTERS 22–25

Ownership and polymorphism

Manage object lifetime and build safe interfaces across class hierarchies.

P135 22.1 / In-lesson ownership defect › Intermediate ☐

Explain and repair a shallow-owning copy

A prototype smart pointer deletes its pointer in its destructor but accepts the compiler-generated copy operation. Reproduce the ownership conflict by inspection and implement one coherent copy policy.

Check With a move-only policy, copying fails at compile time and moving leaves exactly one owner.

Time O(1) for transfer or prohibited copy · **Space** O(1)

P136 22.5 / Q1 › Easy ☐

Give a fraction one owner

Replace a heap-allocated Fraction and its manual delete with make_unique. Keep a nullable, non-owning printing function and pass it access without transferring ownership.

Check Both a live fraction and an empty observer have meaningful print paths.

Time O(1) · **Space** O(1)

P137 22.7 / Q1 › Intermediate ☐

Break a self-owning cycle

A Resource stores a shared pointer to itself, so releasing the external owner never runs its destructor. Change the self-reference inside Resource while preserving the supplied main.

Check A destructor counter increments exactly once at the end of the outer scope.

Time O(1) for one resource · **Space** O(1)

P138 22.x / Q3a › Intermediate ☐

Establish a single shared control block

Fix a program that constructs two independent shared owners from one raw Resource pointer. Construct the resource under one owner and make the second pointer share that ownership.

Check Reset the first owner. The resource survives until the second is reset, then is destroyed once.

Time O(1) · **Space** O(1)

P139 23.3 / Q2 › Intermediate ☐

Aggregate several teachers

Change Department to accept multiple existing Teacher objects and print their names. Preserve identity and ensure those teachers survive department destruction.

Check Create three teachers outside an inner department scope. Use them again after the department is gone.

Time Append amortized O(1), list O(n) plus text output · **Space** O(n) observer handles

CHAPTERS 22–25

Ownership and polymorphism

Manage object lifetime and build safe interfaces across class hierarchies.

P140 23.7 / Q1 › Intermediate ☐

Assign an initializer list to an array

Extend the lesson's owning IntArray with assignment from std::initializer_list<int>. Replace the old length and contents, copy the values into owned storage and return the array for chaining.

Check Assign a shorter list, a longer list and an empty list.

Time O(n) for n new elements · **Space** O(n) replacement storage

P141 24.4 / Q1 › Intermediate ☐

Initialize a fruit base class

Store a name and colour in Fruit. Derive Apple with an additional fibre value and Banana with no additional data. Delegate base initialization correctly and stream each derived object.

Check Stream a const apple and verify its fibre is printed. A banana reports its own variety and colour.

Time O(c) for total string characters · **Space** O(c) owned

P142 24.x / Q2a–b › Intermediate ☐

Specialize an apple variety

Derive Apple and Banana from Fruit. Add GrannySmith as an apple specialization whose construction supplies the appropriate name and green colour. Print them through their intended interfaces.

Check An ordinary apple can still have a different colour from GrannySmith.

Time O(c) for c text characters · **Space** O(c) owned

P143 24.x / Q3a–e › Intermediate ☐

Model a player and monster encounters

Give Creature a name, symbol, health, damage and gold. Derive a levelled Player and a random Monster. Store (health, damage, gold) as dragon (20,4,100), orc (4,2,25) and slime (1,1,10), with symbols D, o and s.

Check A fresh player starts at level 1 with 10 health and 1 damage. A level-up raises both level and damage.

Time Expected O(1) table lookup/random selection with fixed names · **Space** O(1) per entity

P144 24.x / Q3f–g › Advanced ☐

Finish the combat game

Let the player fight or attempt a 50% escape. The player strikes first when fighting. Surviving monsters retaliate. Kills award gold and a level. End at death or level 20. After a win, give a 30% chance to find a potion the player can decline before learning its type. Small/medium/large health potions add 2/2/5 health, strength adds 1 damage and poison subtracts 1 health.

Check Test a failed escape, a killing blow without retaliation, and a potion that changes the outcome.

Time O(t) for t turns with fixed event tables · **Space** O(1) active game state

CHAPTERS 22–25

Ownership and polymorphism

Manage object lifetime and build safe interfaces across class hierarchies.

P145 25.1 / Q1 › Easy ☐

Give base-class observers the correct animal sound

Add a sound member to Animal and initialize it via Cat and Dog constructors. Use the base interface to print each animal's own sound even through a base reference.

Check Accessing the two objects through Animal references produces different sounds.

Time O(1) lookup, O(c) for c characters printed · **Space** O(1) with literal views

P146 25.x / Q1a–f › Intermediate ☐

Diagnose six inheritance defects

Inspect the six linked snippets and make derived behaviour accessible safely. Look for dispatch setup, mismatched const qualification, slicing, a final base, an abstract derived class and deletion through the base type.

Check Build each repaired case separately and verify safe destruction as well as the displayed type.

Time O(1) per fixed example · **Space** O(1)

P147 25.x / Q2a–b › Intermediate ☐

Print a hierarchy of shapes

Define abstract Shape with a virtual destructor and a virtual stream-printing operation. Delegate << to it. Represent a triangle with three points and a circle with one centre and a radius.

Check Stream each through const Shape& and confirm the dynamic shape's geometry appears.

Time O(1) per fixed-size shape · **Space** O(1)

P148 25.x / Q2c–d › Advanced ☐

Search and own heterogeneous shapes

Iterate a collection of shape pointers, print each object and compute the largest circle radius while skipping triangles. Release raw owners, then repeat with a vector of unique_ptr<Shape>.

Check Two circles of radii 9 and 4 with a triangle between them yield 9.

Time O(n) for n fixed-size shapes · **Space** O(n) owners, O(1) scan

CHAPTERS 26–28, C

Templates, exceptions and I/O

Specialize types, report failure and complete the course with a larger program.

P149 26.x / Q1–2 › Intermediate ☐

Design homogeneous and mixed pairs

Implement Pair1<T> with two fields of one type, construction and const access. Then implement Pair<T,U> so the two field types can differ.

Check Read an integer pair and a string/double pair using the same accessor names.

Time O(1) access · **Space** Storage and copying follow the field types

P150 26.x / Q3 › Intermediate ☐

Fix the first type of a pair

Derive StringValuePair<T> from Pair<std::string,T>. Forward construction into the base and reuse its accessors.

Check A key and a fractional value round-trip through a const object.

Time O(k) to copy a key of length k for scalar T · **Space** O(k) owned

P151 27.x / Q1 › Intermediate ☐

Reject a zero denominator

In the Fraction constructor, throw std::runtime_error for a zero denominator. Read two integers in main, print a valid fraction or catch the exception through a const std::exception reference and report the error.

Check 9/4 succeeds and 9/0 takes the catch path without creating a Fraction.

Time O(1) numeric work · **Space** O(1) for fixed diagnostics

CHAPTERS 26–28, C

Templates, exceptions and I/O

Specialize types, report failure and complete the course with a larger program.

P152 28.5 / Guided stream lab › Intermediate ☐

Validate a complete numeric input

Read a line as text and parse a number only if the whole accepted field is valid. Handle malformed text, extra characters and the stream running out of data.

Check A clean integer succeeds. A number followed by letters fails complete-field validation.

Time O(b) for b characters · **Space** O(b) line buffer

P153 28.6 / Guided file lab › Easy ☐

Write, append and read a text file

Write two lines to a file, reopen in append mode to add a third, then read it one full line at a time. Check each file-open result.

Check Preserve internal spaces and verify all three lines arrive in order.

Time O(b) for b file characters · **Space** O(L) for the longest line

P154 28.7 / Guided file lab › Intermediate ☐

Edit bytes in place

Open a sample file for reading and writing in binary mode. Replace each ASCII vowel with # at the same byte position while preserving the rest of the file. Reposition the stream correctly when changing direction.

Check “area 51” becomes “#r## 51” and the file length stays the same.

Time O(b) for b byte visits, excluding device-specific seek latency · **Space** O(1)

P155 C.1 / Closing project suggestion › Advanced ☐

Build a larger console game

Choose a small simulation or card game, such as the suggested Crazy Eights. Define the rules you will support, implement a playable loop and refine a measured bottleneck after the game works.

Check For a card game, use a fixed deck order to replay one entire legal game and its end condition.

Time Design-dependent. A hand scan takes O(h) per move for h cards · **Space** O(d) for d cards in play

SOURCE DIRECTORY / 1 OF 3

The course at a glance

Choose a lesson suffix on its chapter row to open the original page.

■ Has a workbook entry ☐ Other reviewed course page

Chapter 0	Getting started	1 2 3 4 5 6 **7** 8 9 10 11 12 **13**
Chapter 1	C++ basics	1 2 3 4 **5** 6 7 8 9 10 **11** **x**
Chapter 2	Functions and files	1 2 3 **4** 5 6 **7** **8** 9 10 11 **12** 13 **x**
Chapter 3	Debugging	1 2 3 4 5 **6** **7** **8** **9** 10 **x**
Chapter 4	Fundamental types	1 2 3 4 5 6 7 8 9 **10** 11 **12** **x**
Chapter 5	Constants and strings	1 2 3 4 5 6 **7** 8 9 **x**
Chapter 6	Operators	**1** 2 **3** 4 5 6 7 **8** **x**
Chapter 0	Bit manipulation	1 **2** **3** **4**
Chapter 7	Scope and linkage	1 2 **3** 4 5 6 7 8 9 10 11 12 13 14 **x**
Chapter 8	Control flow	1 2 3 4 5 **6** 7 **8** 9 **10** 11 12 13 **14** 15 **x**
Chapter 9	Error handling	1 **2** 3 4 5 6 **x**
Chapter 10	Conversions and deduction	1 2 3 4 5 6 **7** 8 9 **x**

SOURCE DIRECTORY / 2 OF 3

The course at a glance

Choose a lesson suffix on its chapter row to open the original page.

■ Has a workbook entry ☐ Other reviewed course page

Chapter 11 — Function templates
1 2 3 4 5 6 7 8 **9** **10** **x**

Chapter F — Constexpr functions
1 2 3 4 **X**

Chapter 12 — References and pointers
1 2 3 4 5 6 **7** 8 9 10 11 12 13 14 15 **x**

Chapter 13 — Enums and structs
1 **2** 3 4 5 **6** 7 8 9 **10** 11 12 13 14 15 **x** **y**

Chapter 14 — Introduction to classes
1 2 **3** 4 **5** 6 7 8 9 **10** 11 **12** 13 14 15 16 17
x

Chapter 15 — More on classes
1 2 3 4 **5** 6 **7** 8 **9** 10 **x**

Chapter 16 — Dynamic arrays
1 **2** **3** **4** 5 **6** **7** **8** **9** 10 **11** 12 **x**

Chapter 17 — Fixed-size arrays
1 **2** **3** **4** 5 **6** **7** 8 9 **10** 11 12 13 **x**

Chapter 18 — Iterators and algorithms
1 2 3 **4**

Chapter 19 — Dynamic allocation
1 **2** 3 4 5

Chapter 20 — Functions
1 2 **3** 4 5 **6** **7** **x**

Chapter 21 — Operator overloading
1 **2** 3 **4** 5 **6** **7** 8 **9** **10** 11 12 13 14 **x** **y**

SOURCE DIRECTORY / 3 OF 3

The course at a glance

Choose a lesson suffix on its chapter row to open the original page.

■ Has a workbook entry ☐ Other reviewed course page

Chapter 22	Move and smart pointers	**1** 2 3 4 **5** 6 **7** **x**
Chapter 23	Object relationships	1 2 **3** 4 5 6 **7** x
Chapter 24	Inheritance	1 2 3 **4** 5 6 7 8 9 **x**
Chapter 25	Virtual functions	**1** 2 3 4 5 6 7 8 9 10 11 **x**
Chapter 26	Templates and classes	1 2 3 4 5 6 **x**
Chapter 27	Exceptions	1 2 3 4 5 6 7 8 9 10 **x**
Chapter 28	Input and output	1 2 3 4 **5** **6** **7**
Appendix A	Miscellaneous subjects	1 2 3 4
Appendix B	Standard updates	1 2 3 4 5
Appendix C	Further practice	**1**
Appendix D	Deprecated articles	21.1 21.2 21.3 21.4 22.1 22.2 22.3 22.4 22.5 22.6 22.7